DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING

OBJECT ORIENTED PROGRAMMING

FILE STORAGE SYSTEM
Project Report

SUBMITTED TO:

Lec. Anum Abdul Salam
LE Kashaf Raheem

SUBMITTED BY:

Abdur Rehman – 462554
Fatima Masood – 456153
Roma Manahil – 482336

Submission Date: 14/5/25

CONTENTS

Introduction 1

Problem Statement 2

Objectives 3

System Design 4

Class Diagram 5

System Level Diagram 6

Implementation Details 7

Key Features 8

Results & Outputs 9

Challenges & Solutions 10

Conclusion 11

References 12

➢ Introduction:

The File Storage System project aims to design and implement a virtual hard disk that simulates the functionality of a file system within a 10 MB file. This system is structured into three distinct portions: a metadata section for file names and addresses, a section for tracking available blocks, and a data storage section. The system supports essential file operations such as creating, listing, copying, modifying, and deleting files, along with an advanced defragmentation feature to optimize storage efficiency.

This project addresses the need for a compact, efficient, and user-friendly file management system that can handle dynamic data storage requirements while minimizing fragmentation. By loading data into RAM during execution and updating the disk only upon termination, the system ensures optimal performance and data integrity.

➢ Problem Statement:

Modern computing systems require efficient file management solutions to handle data storage, retrieval, and modification. However, implementing a file system from scratch poses challenges, including:

1. **Memory Management:** Allocating and deallocating blocks dynamically while avoiding fragmentation.
2. **File Operations:** Supporting operations like creation, deletion, copying, and modification while maintaining data consistency.
3. **Performance Optimization:** Ensuring quick access to files and efficient use of storage space.
4. **User Interaction:** Providing a seamless interface for users to perform file operations without complexity.

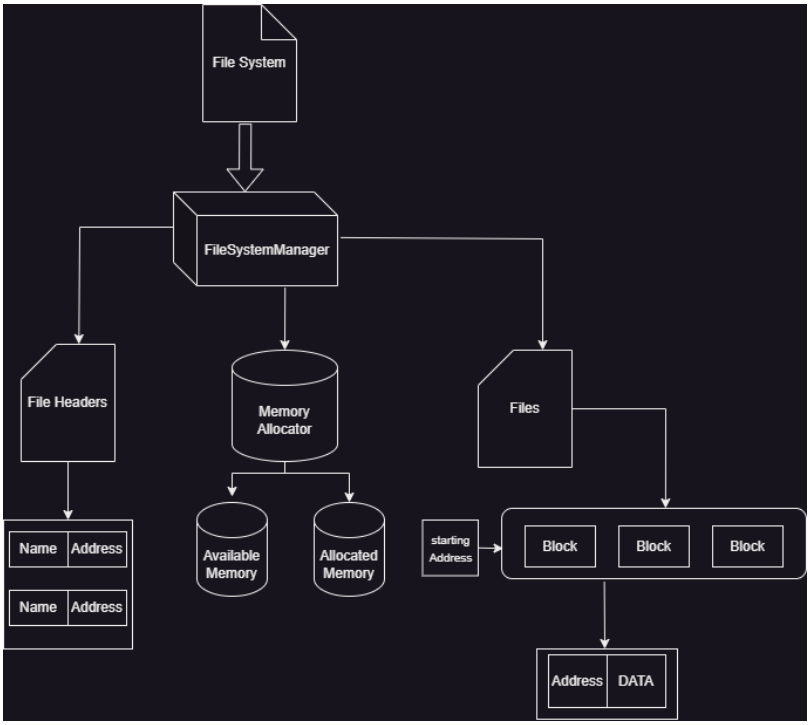
This project tackles these challenges by designing a structured file system within a 10 MB file, adhering to strict specifications for metadata, block allocation, and data storage.

➢ Objectives:

- To implement a virtual 10MB file storage system simulating disk operations.
- To design an OOP-based scalable architecture for file management.
- To manage metadata (1MB), block allocation (1MB), and data storage (8MB) partitions.
- To support dynamic file operations (create, read, modify, delete) with collision handling.
- To enable cross-system file transfers between virtual storage and host OS.
- To implement defragmentation for storage optimization (bonus feature).
- To provide GUI-based interaction with real-time system monitoring.
- To maintain data persistence through RAM caching with disk synchronization.
- To analyze system performance through operation logging and metrics.

➢ System Design:

The system design is based on an Data Structure approach, ensuring modularity and scalability.



➢ Implementation Details:

- Programming Language: C++.

1. HeaderFiles:

- **Purpose:** Manages file names and their starting addresses.
- Uses a map (fileMap) to store filenames and addresses.
- addFile(), deleteFile(), checkName(), getAdress(), display(): Functions to manage and view file metadata.
- Stores data in 500B entries.

2. Files:

- **Purpose:** Manages the actual file data.
- Uses a map (files) to store file data in deques of Blocks.
- Block struct stores block address and data.
- Functions to appendToFile(), createNewFile(), removeFromEnd(), deleteFile(), getFile(), and display() file content.

3. FileSystemManager

- **Purpose:** Orchestrates file system operations.
- Contains instances of HeaderFiles, Files, and MemoryAllocator.
- Functions to addFile(), listFiles(), viewFile(), deleteFile(), copyFileFromWindows(), copyFileToWindows(), and Modify() files.
- Implements Defragmentation().
- Constructor and destructor handle loading/saving data.

4. MemoryAllocator

- **Purpose:** Manages allocation/deallocation of data blocks.
- Uses a map (AllocatedMemory) and a priority_queue (FreeMemory) to track block availability.
- Functions to allocateBlock(), deallocateBlock(), and isAvailable().
- Allocates blocks of BLOCK_SIZE (1024 bytes).

Key Classes:

1-Files

2-File Header

3-Memory Allocator

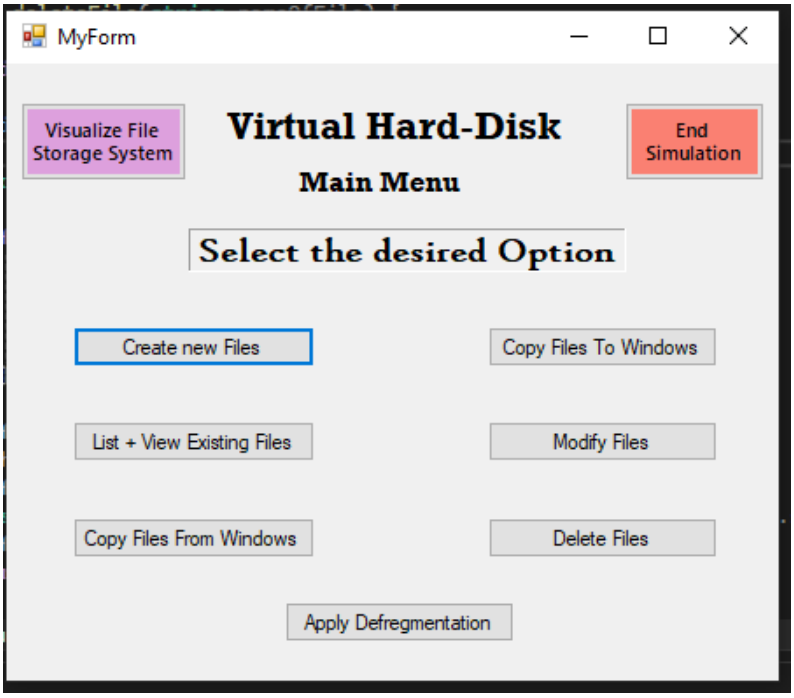
4- File System Manager

➢ Key Features:

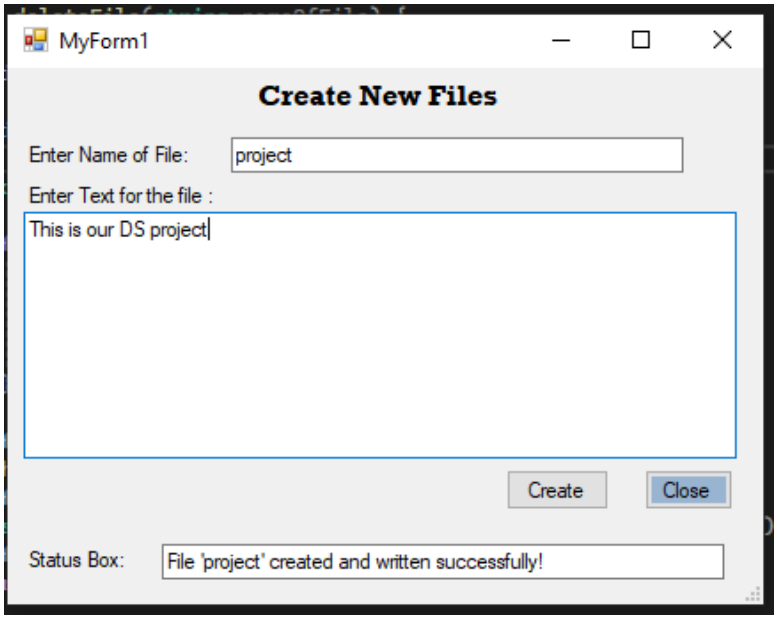
- Simulates a 10MB file storage system.
- Supports Create, List & View, Copy (to/from Windows *.txt), Modify, and Delete file operations.
- The virtual disk is divided into sections for metadata, free space management, and data.
- Stores file names and starting addresses in 500B blocks.
- Allocates and manages data blocks for file content within the 8MB data area.
- Loads data on startup and saves changes upon program termination.
- Optionally includes a defragmentation feature.
- The system is designed to work with *.txt files.

➢ Results & Output:

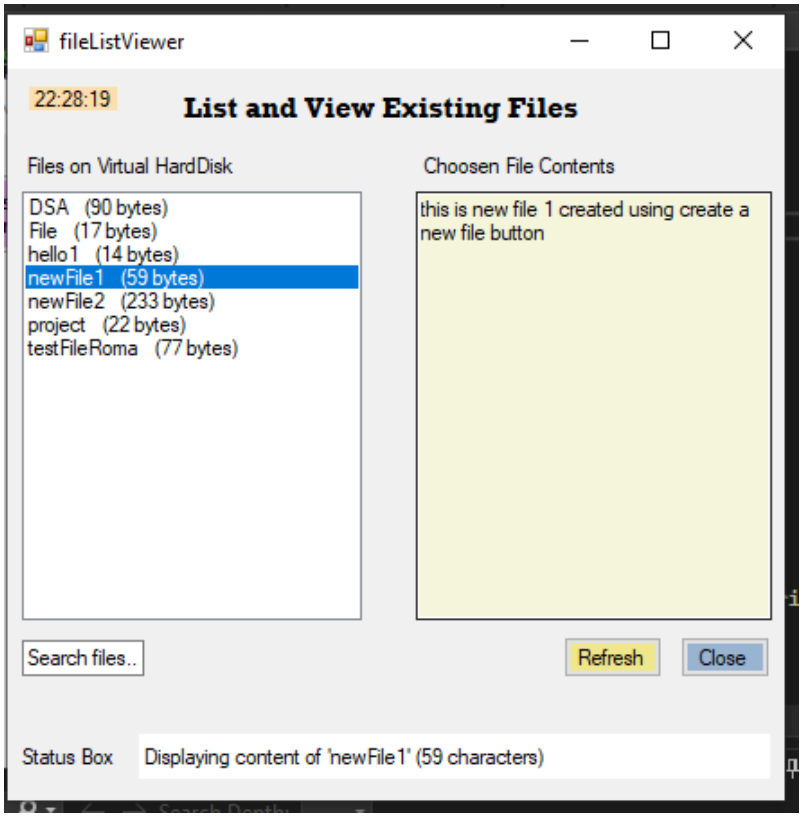
Main Form:



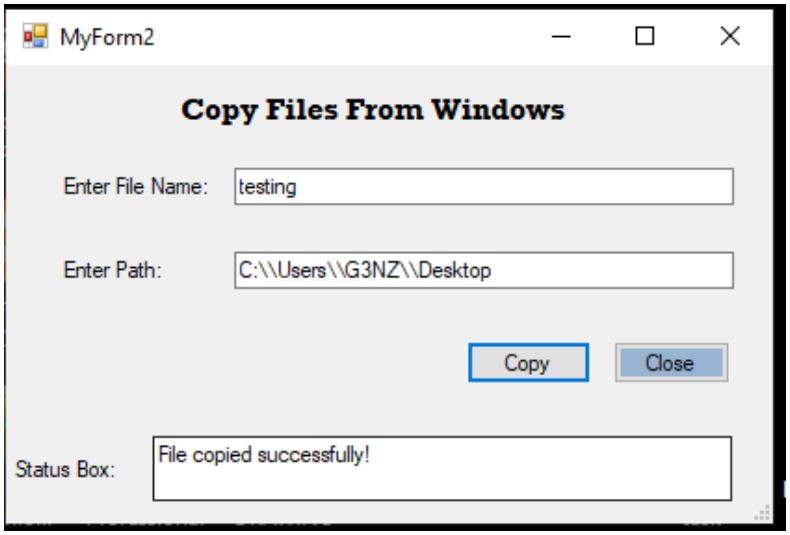
Create New File:



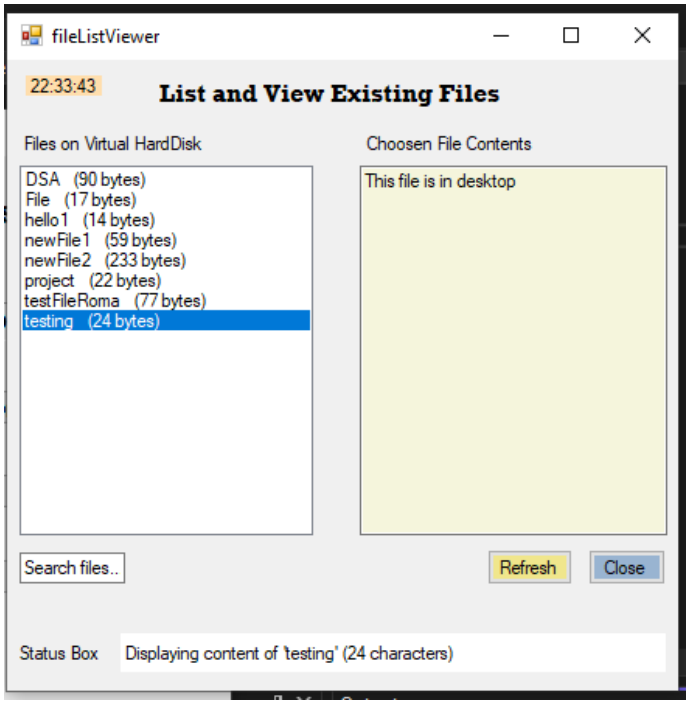
List & View Existing Files:



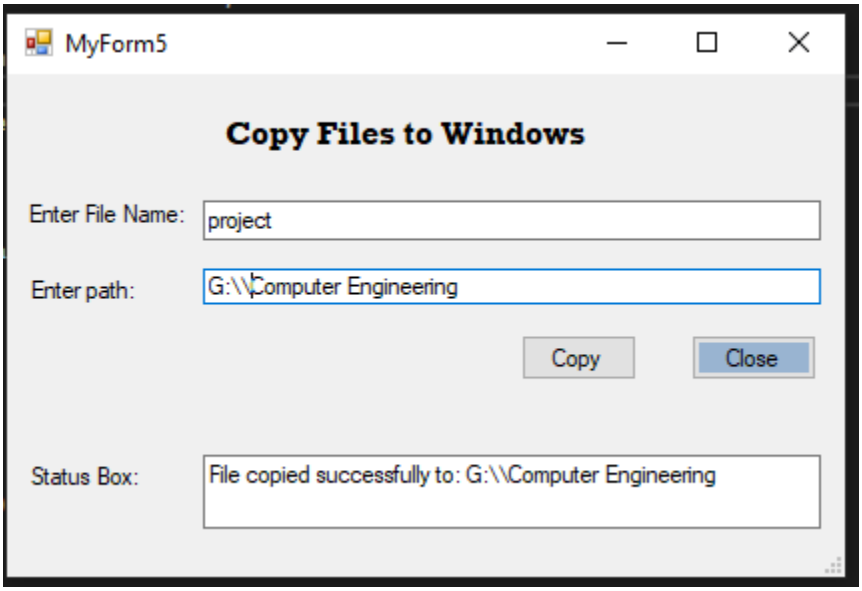
Copy Files From Windows:



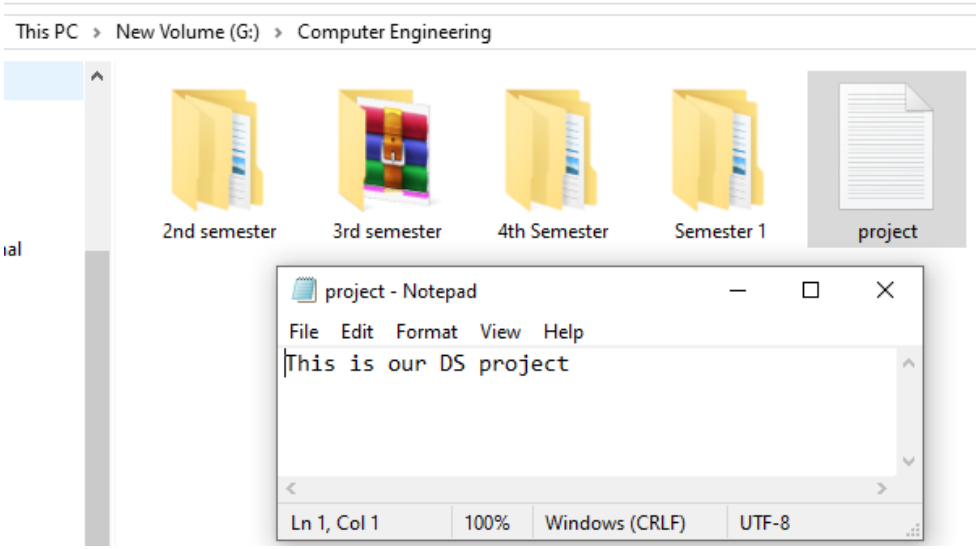
File is successfully copied from desktop to our file system(virtual harddisk).



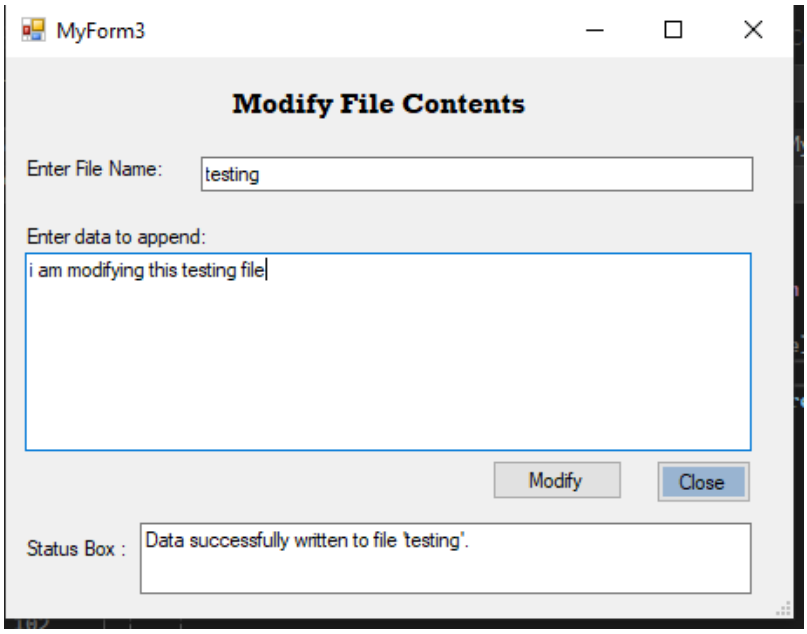
Copy Files To Windows:



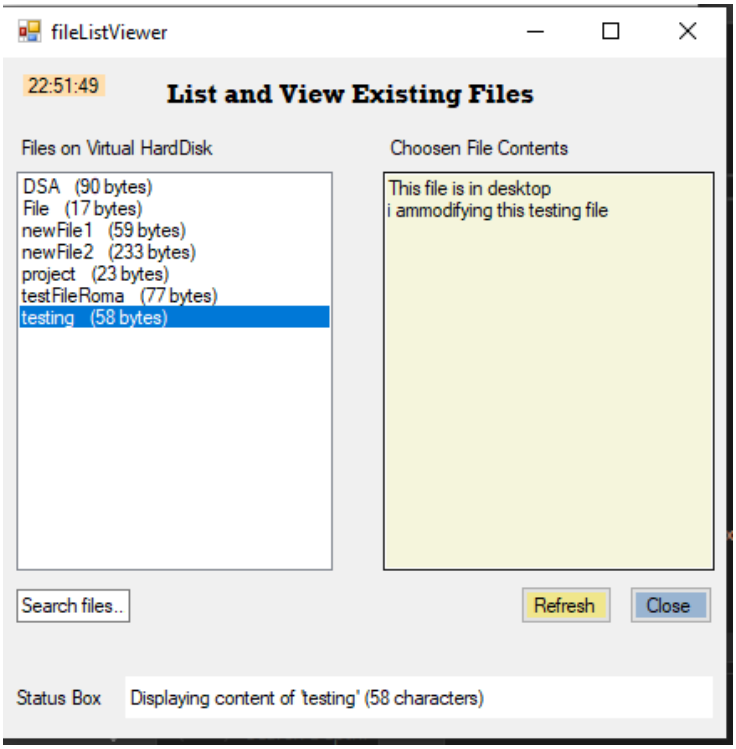
File is successfully copied from our file system(virtual harddisk) to our windows(New Volume G:).



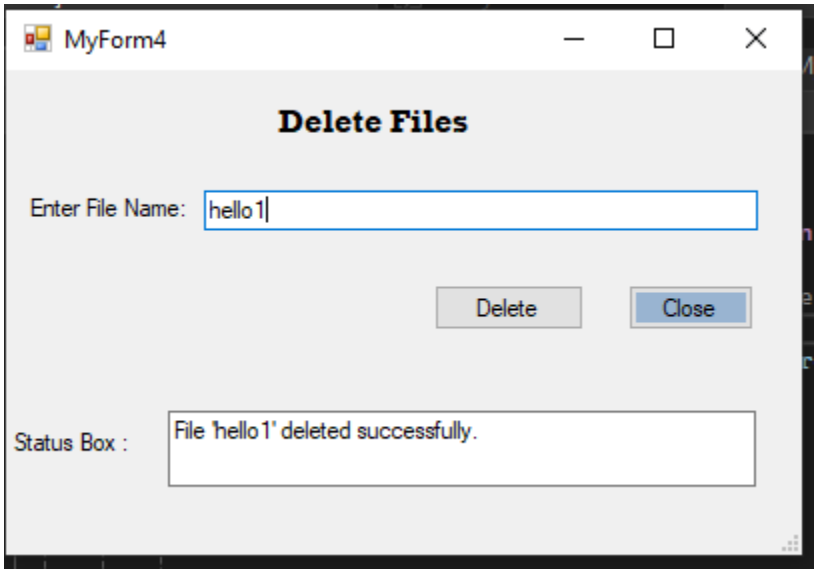
Modify File:



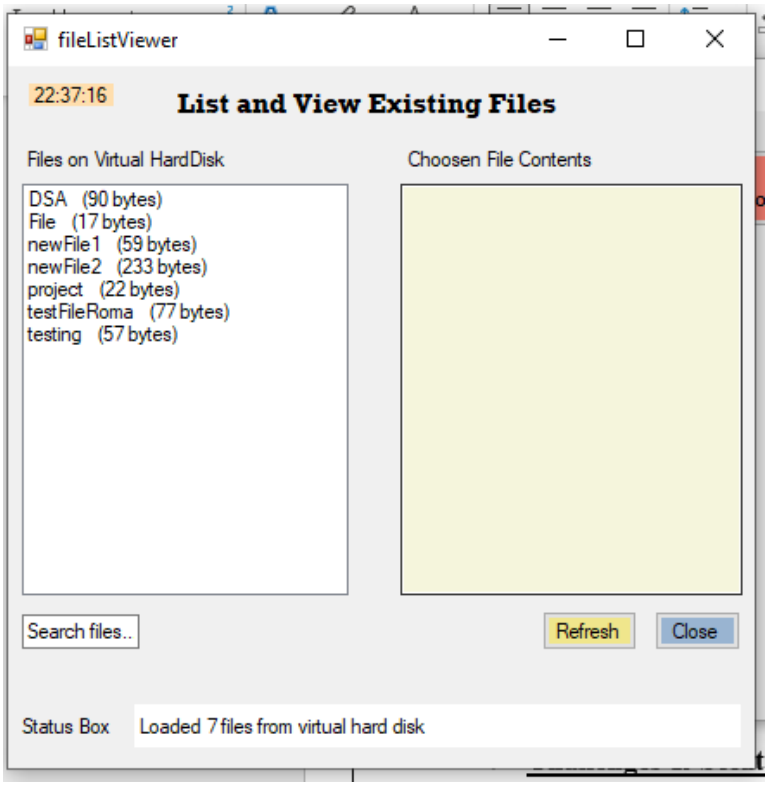
Modified text of file “testing” can be seen.



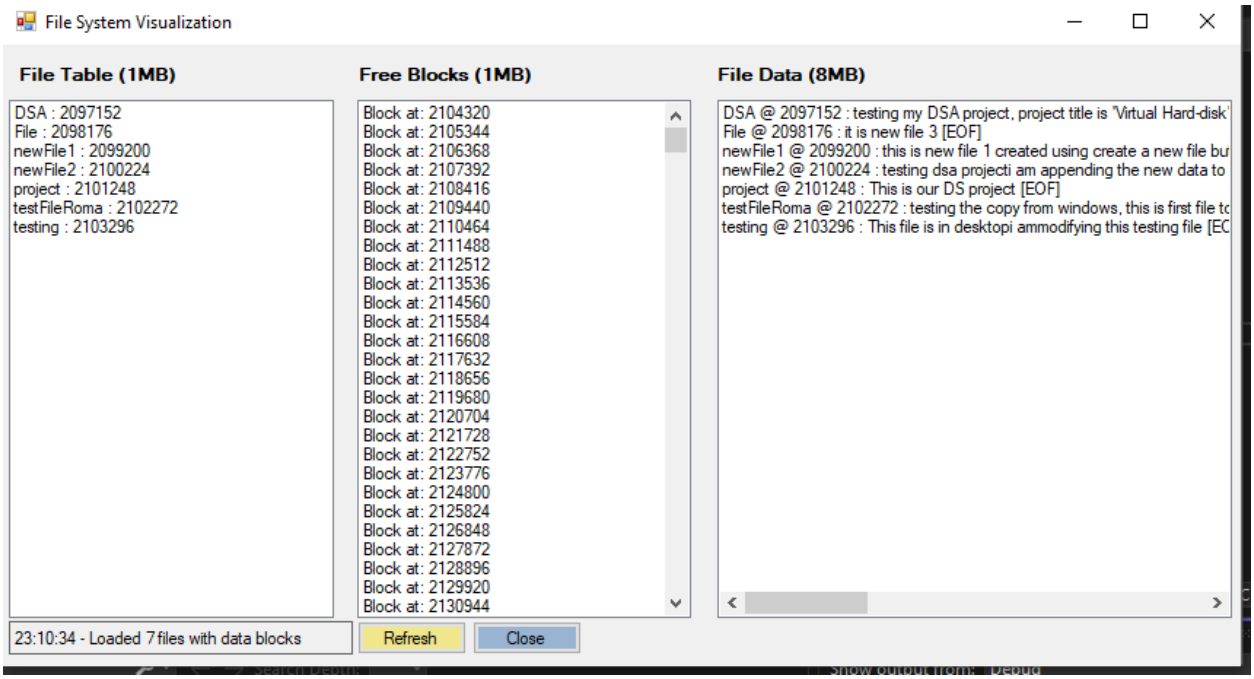
Delete File:



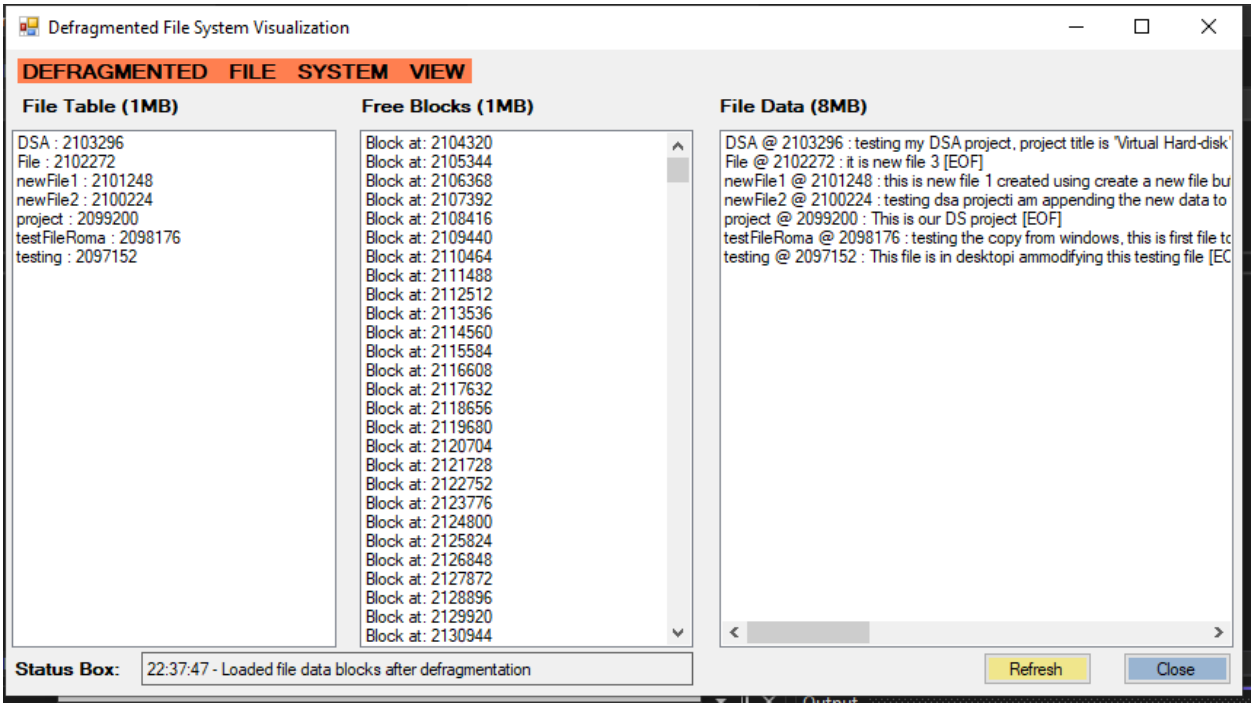
hello1 is successfully deleted and can be seen in list of our existing files.



Visualize File System:

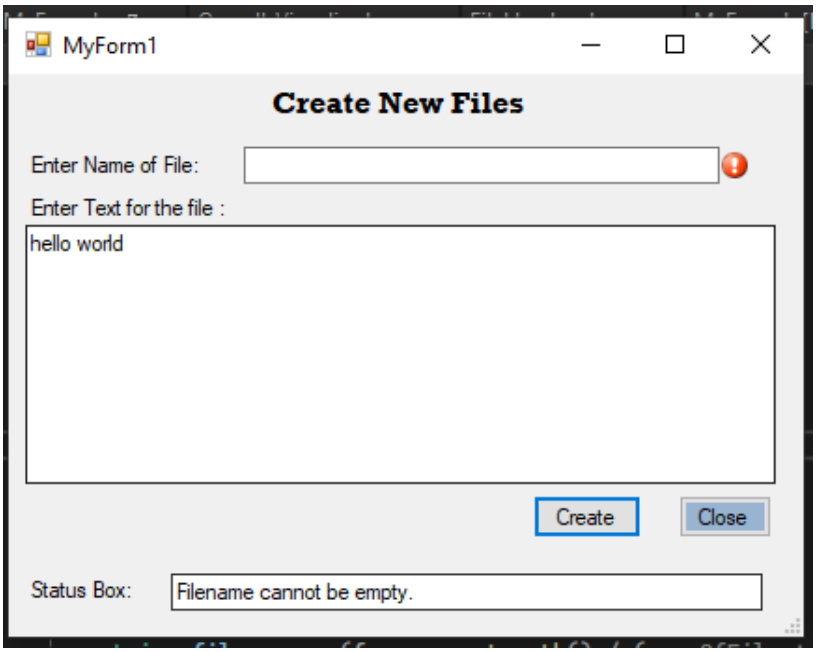


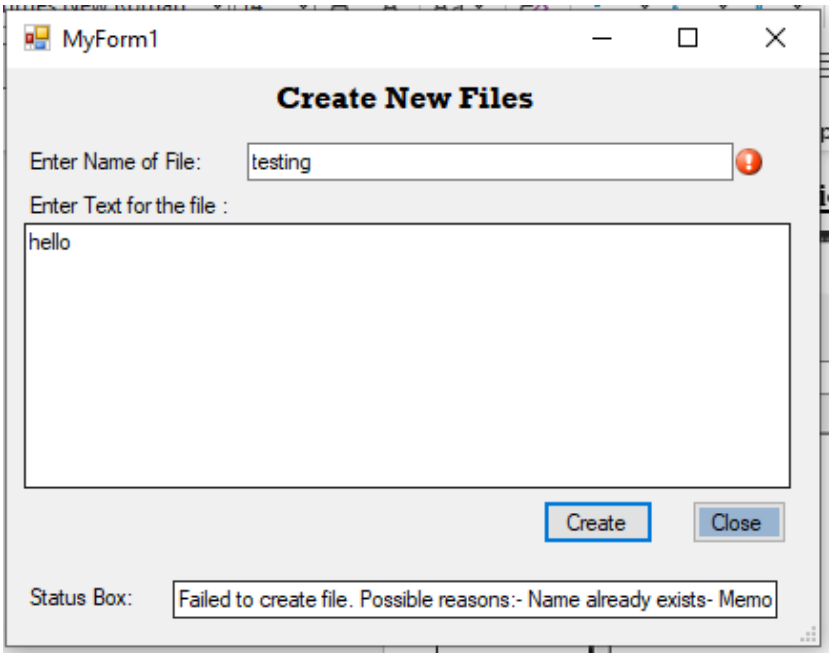
Defregmentation:



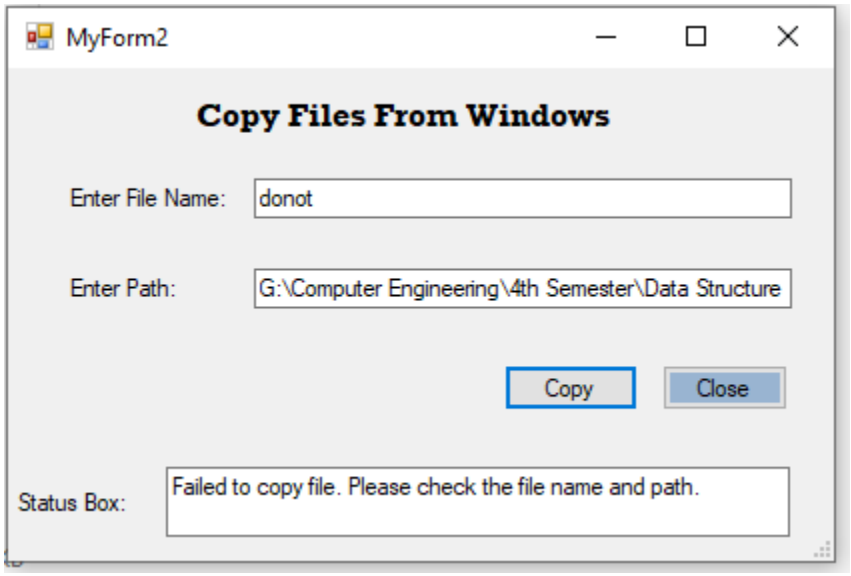
➢ Error Validation:

Create New File:

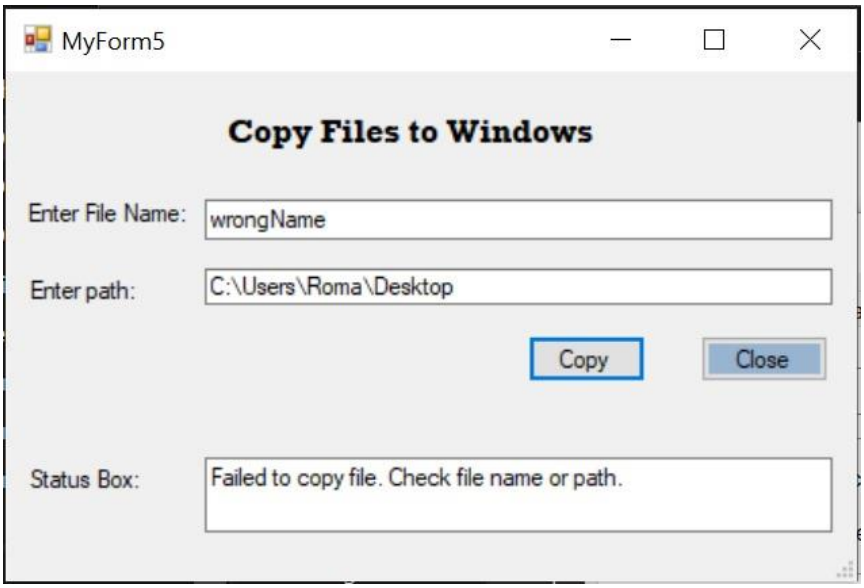




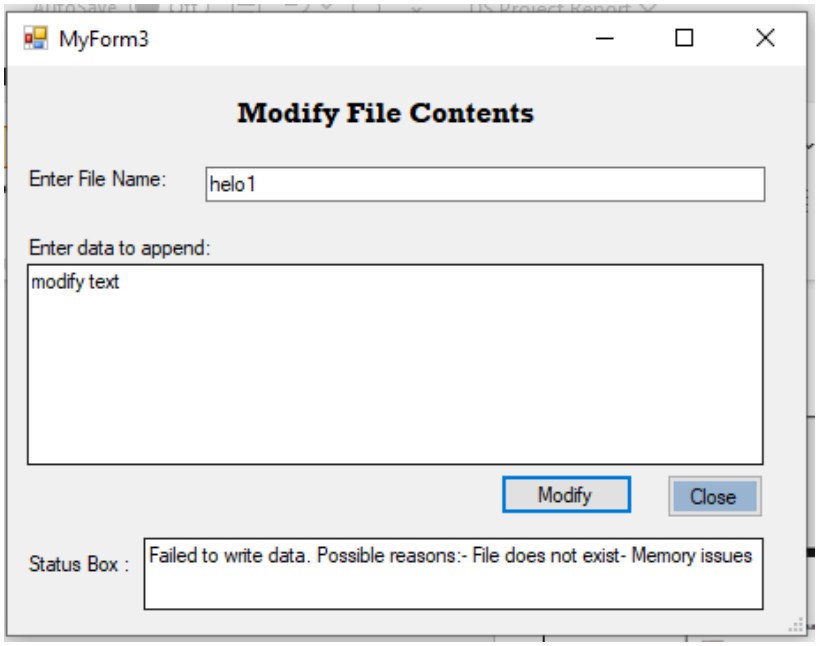
Copy File From Windows:



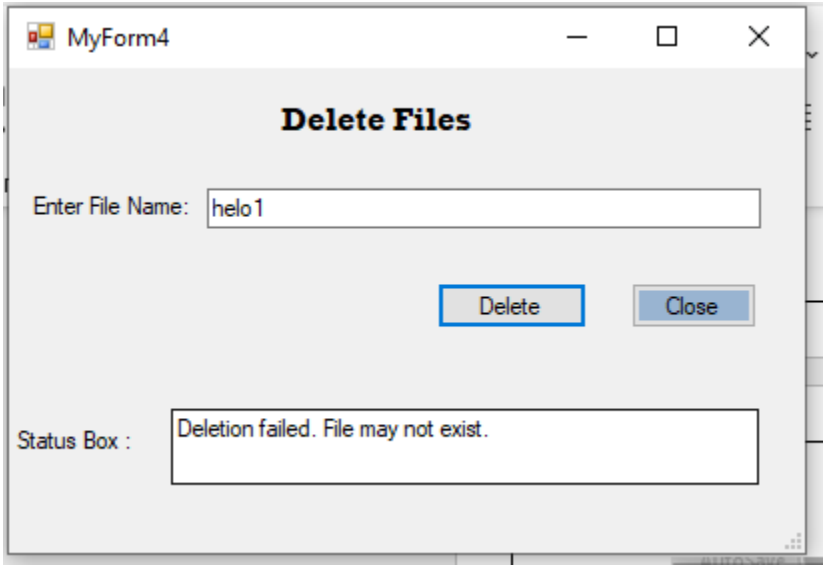
Copy File To Windows:



Modify File:



Delete File:



Challenges & Solution:

- **Challenge:** Handling files larger than a single block (1024B).
  - **Solution:** Implementation of linked blocks, where each block stores the address of the next block in the file.
- **Challenge:** Efficiently tracking and allocating free space.
  - **Solution:** Using the MemoryAllocator class with a priority queue (FreeMemory) to manage free block addresses.
- **Challenge:** Ensuring data integrity during file operations.
  - **Solution:** Using a delimiter to mark the end of data within a block and careful management of block addresses.
- **Challenge:** Implementing file defragmentation.
  - **Solution:** Employing a recursive algorithm (recursiveFragmentation) to rearrange file blocks.
- **Challenge:** Optimizing file I/O operations.
  - **Solution:** Loading the virtual disk into memory at startup and saving it at termination.

Conclusion:

This project successfully implements a **virtual file storage system** with a structured 10 MB disk layout, supporting essential file operations and advanced defragmentation. Key achievements include:

- Efficient block management for dynamic file storage.
- Seamless file operations (create, read, modify, delete, copy).
- Defragmentation to optimize storage performance.
- User-friendly console interface for easy interaction.

Future developments could include:

- Enhanced Defragmentation: Implement more advanced defragmentation algorithms to minimize disk movement and improve performance.

- Directory Structure: Add support for directories and subdirectories to organize files hierarchically, rather than just a flat file structure.
- Support for Other File Types: Extend the system to support file types other than .txt, such as binary files.
- Caching: Implement a caching mechanism to store frequently accessed data in memory for faster retrieval.
- Compression: Add file compression to reduce storage space requirements.